

计算机系统结构实验报告Lab04: Reg, Data Mem, Signext

uu-matter543

2024-2025-2

摘要

在 Lab04 中，我成功实现了 MIPS 32 单周期处理器的 3 个部件：Registers, Data Memory, Sign Extend，学习并使用了 Verilog 的位组合逻辑，进一步了解了 CPU 的模块化设计以及利用状态机思想设计 CPU 模块，收获颇丰。

目录

1	实验目的	2
2	原理分析	2
2.1	Registers 实验分析	2
2.2	Data Memory 原理分析	2
2.3	Sign Extend 原理分析	2
3	功能实现	2
3.1	Registers 功能实现	2
3.2	Data Memory 功能实现	3
3.3	Sign Extend 功能实现	4
4	功能仿真	4
4.1	Registers 功能仿真	4
4.2	Data Memory 功能仿真	6
4.3	Sign Extend 功能仿真	8
5	总结与反思	8

1 实验目的

1. 理解寄存器、数据存储器、有符号扩展单元的 IO 定义；
2. Registers 的设计实现；
3. Data Memory 的设计实现；
4. Sign Extend 的设计实现；
5. 对功能模块进行仿真。

2 原理分析

2.1 Registers 实验分析

Registers 由 32 个寄存器组成，每个寄存器的位宽为 32 位。寄存器组有 2 个读端口，根据 2 个 5 位读寄存器选择信号，可以同时读出 2 个寄存器的 32 位数据；1 个写端口，在写使能信号为 1 的时钟下降沿，将数据写入寄存器。由于不确定写寄存器选择 (wr_reg) 信号、写入数据 (wr_data) 信号、寄存器写使能 (reg_wr_en) 信号的先后次序，采用下降沿写入的方式，以免发生错误和冒险等情况。

2.2 Data Memory 原理分析

Data Memory 即数据内存，位宽也是 32 位。Data Memory 的读和写都需要使能信号，读仍然是异步的，写在时钟下降沿进行。一方面，可能实际内存时钟与处理器时钟并不同步；另一方面，内存速度比较慢，不可能在 1 个处理器时钟周期内完成写入。从处理器视角，本实验的“内存”其实是缓存。实际上，本实验中的 Data Memory 用较少量 (64个) 的寄存器模拟，区别仅在于读取也需要使能信号，以防止访问非法地址。

2.3 Sign Extend 原理分析

Sign Extend 接受 16 位 Immediate Number，根据补码表示数原理，将其符号位扩展为 32 位，即将符号位复制到高 16 位。后续实验可能需要 Zero Extend，其设计原理与 Sign Extend 类似，只需要修改扩展逻辑即可。

3 功能实现

3.1 Registers 功能实现

```
module registers(  
    input clock,  
    input [25:21] rd_reg_1,  
    input [20:16] rd_reg_2,  
    input [15:11] wr_reg,  
    output reg [31:0] rd_data_1,  
    output reg [31:0] rd_data_2,  
    input [31:0] wr_data,  
    input wr_en  
);  
reg [31:0] reg_cont [31:0];  
integer i;  
initial for (i = 0; i < 32; i = i + 1) reg_cont[i] <= 0;  
always @ (rd_reg_1 or rd_reg_2) begin  
    rd_data_1 = reg_cont[rd_reg_1];  
    rd_data_2 = reg_cont[rd_reg_2];  
end  
always @ (negedge clock)  
    if (wr_en) reg_cont[wr_reg] <= wr_data;  
endmodule
```

定义时钟端口以实现下降沿写入；定义 2 个读寄存器端口和 1 个写寄存器端口，以确定读写操作的目标寄存器；定义 2 个读数据端口和 1 个写数据端口；定义写使能信号端口。在内部定义 32 个 32 位寄存器以实现寄存器的存储功能。

初始化寄存器的值均为 0。根据实验指导书，在读端口语句块中添加端口值更新的语句；在写端口语句块中添加时钟下降沿写数据逻辑。

3.2 Data Memory 功能实现

```
module data_memory(  
    input clock,  
    input [31:0] addr,  
    input mem_wr,  
    input [31:0] wr_data,  
    input mem_rd,  
    output [31:0] rd_data  
);  
reg [31:0] mem_cont [63:0];  
integer i;
```

```
initial for(i = 0; i < 64; i = i + 1) mem_cont[i] <= 0;
assign rd_data = mem_rd && !mem_wr ? mem_cont[addr] : 0;
always @ (negedge clock) if (mem_wr) mem_cont[addr] = wr_data;
endmodule
```

定义时钟端口以实现下降沿写入；定义读地址端口和写地址端口，以确定读写操作的目标地址；定义读数据端口和写数据端口；定义读使能信号和写使能信号端口。在内部定义 64 个 32 位寄存器以实现数据内存的存储功能。

初始化寄存器的值均为 0。根据实验指导书，在读端口语句块中添加端口值更新的语句，注意添加使能控制逻辑：仅在读使能信号为 1 且写使能信号为 0 时允许读取内存数据；在写端口语句块中添加时钟下降沿写数据逻辑。

3.3 Sign Extend 功能实现

```
module sign_extend(
    input  [15:0] data_16,
    output [31:0] data_32
);
    assign data_32 = {{16{data_16[15]}} , data_16};
endmodule
```

位扩展功能实现比较简单，只需要将输入 16 位数据的符号位复制到输出 32 位数据的高 16 位即可实现位扩展。

4 功能仿真

依然要注意在进行功能仿真时，要将需要仿真的模块对应的仿真激励 Set as Top。

4.1 Registers 功能仿真

编写激励文件 registers_tb.v，连接仿真激励输入与模块输入，在每个要仿真的功能之间添加时延，以便在波形图上观察到输出信号的变化，主要的仿真激励如下：

```
always #(100) Clock = !Clock;
// negedge at 100, 300, 500, 700, etc.

initial begin
    Clock = 1;
    ReadReg1 = 0;
```

```
ReadReg2 = 0;
WriteReg = 0;
WriteData = 0;
WriteEnable = 0;

#250; //Time 250
WriteEnable = 1;
WriteReg = 21;
WriteData = 32'hffff0000;
//At 300 Write Operation taken

#200; //Time 450
WriteReg = 10;
WriteData = 32'h0000ffff;
//At 500 Write Operation taken

#200; //Time 650
WriteEnable = 0;
WriteReg = 0;
WriteData = 32'hffffffff;
//Write Operation will be banned by WriteEnable=0

#75; //Time 725
ReadReg1 = 21;
ReadReg2 = 10;
//Read data written by operation at 300 & 500

#75; //Time 800
ReadReg1 = 0;
ReadReg2 = 0;
//Confirm that operation at 700 is banned
end
```

仿真输入与寄存器组的期望反应为：

1. 初始化时钟周期为 200ns，且在 $t = 100(2k + 1)$, $k = 0, 1, 2, 3, \dots$ 时为下降沿；
2. 在 250 ~ 450ns 向 21 号寄存器写 *ffff0000*，寄存器组会在 300 时刻将其写入；
3. 在 450 ~ 650ns 向 10 号寄存器写 *0000ffff*，寄存器组会在 500 时刻将其写入；
4. 在 650 ~ 725ns，将目标寄存器设为 0 号寄存器，将写入数据设为 *ffffffff*，但是

关闭写使能信号，寄存器组不会做任何操作；

- 在 725 ~ 800ns，2 个读寄存器端口分别读取 21 号寄存器和 10 号寄存器的数据，寄存器读数据端口应分别输出 *ffff0000* 和 *0000ffff*；
- 在 800 时刻以后，读取 0 号寄存器的数据，由于 $t=700\text{ns}$ 时的写入无效，读数据端口应为 *00000000*，以验证写使能信号的有效性。

仿真波形如下：

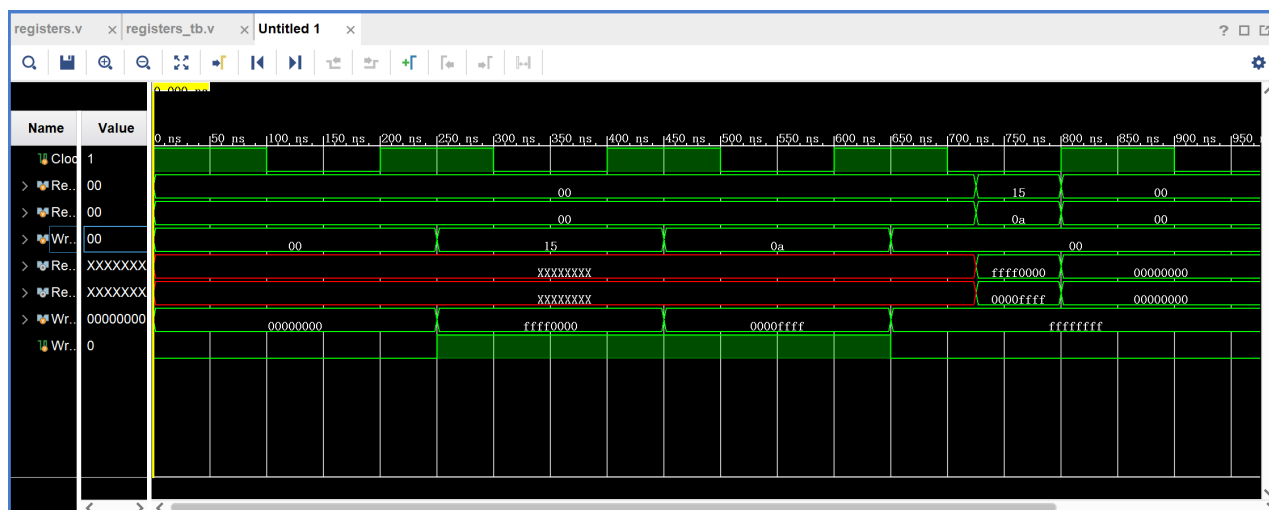


图 1: Registers 功能仿真

核对波形无误，说明 Registers 功能已实现。

4.2 Data Memory 功能仿真

编写激励文件 `data_memory_tb.v`，连接仿真激励输入与模块输入，在每个要仿真的功能之间添加时延，以便在波形图上观察到输出信号的变化，主要的仿真激励如下：

```
always #(100) Clock = !Clock;

initial begin
    Clock = 0;
    Address = 0;
    WriteData = 0;
    MemoryWrite = 0;
    MemoryRead = 0;

    #185; //Time 185
    MemoryWrite = 1;
    Address = 32'b000111;
```

```
WriteData = 32'he0000000;  
  
#100; //Time 285  
MemoryWrite = 1;  
WriteData = 32'hfffffff;  
Address = 32'b000110;  
  
#185; //Time 470  
Address = 7;  
MemoryRead = 1;  
MemoryWrite = 0;  
  
#80; //Time 550  
MemoryWrite = 1;  
Address = 8;  
WriteData = 32'haaaaaaaa;  
  
#80; //Time 630  
Address = 6;  
MemoryWrite = 0;  
MemoryRead = 1;  
  
end
```

仿真输入与实验指导书一致，因此仿真波形应与实验指导书一致。仿真波形如下：

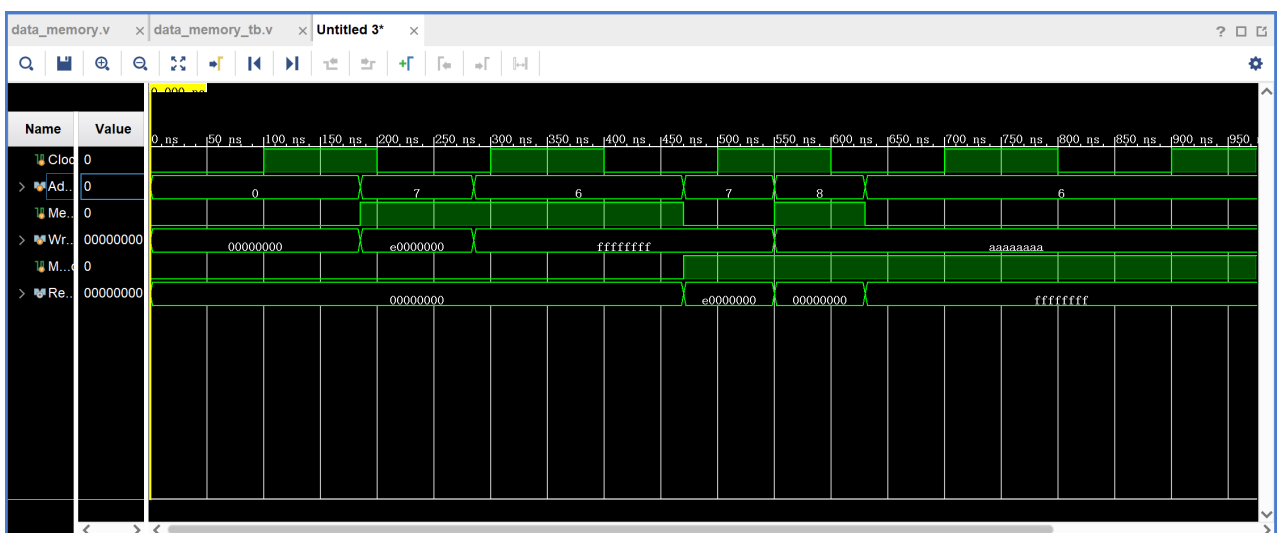


图 2: Data Memory 功能仿真

与实验指导书给出的波形一致，说明 Data Memory 功能基本实现。

4.3 Sign Extend 功能仿真

编写激励文件 `sign_extend_tb.v`, 连接仿真激励输入与模块输入, 在每个要仿真的功能之间添加时延, 以便在波形图上观察到输出信号的变化, 主要的仿真激励如下:

```
initial begin
    Data16 = 0;
    #100; Data16 = 16'h0001;
    #100; Data16 = 16'hffff;
    #100; Data16 = 16'h7fff;
    #100; Data16 = 16'h8000;
end
```

分别选取 0000, 0001, ffff, 7fff, 8000 作为 16 位输入, 分别得到 32 位输出, 仿真波形如下页图所示。与预期结果一致, 说明 Sign Extend 功能基本实现。

5 总结与反思

在 Lab04 中, 我实现了以上 3 个模块。Lab03 和 Lab04 已经将处理器的大部分组成模块实现, 后续只要连接内部数据通路就可以实现一个简单的单周期处理器。

实验中有几个值得注意的地方:

1. 读取和写入在什么时候进行;
2. 如何初始化;
3. 怎么拼接数字;

在后续实验实现处理器时, 应对这些问题加以注意, 以应对指令执行过程中信号传递逻辑出现的问题。

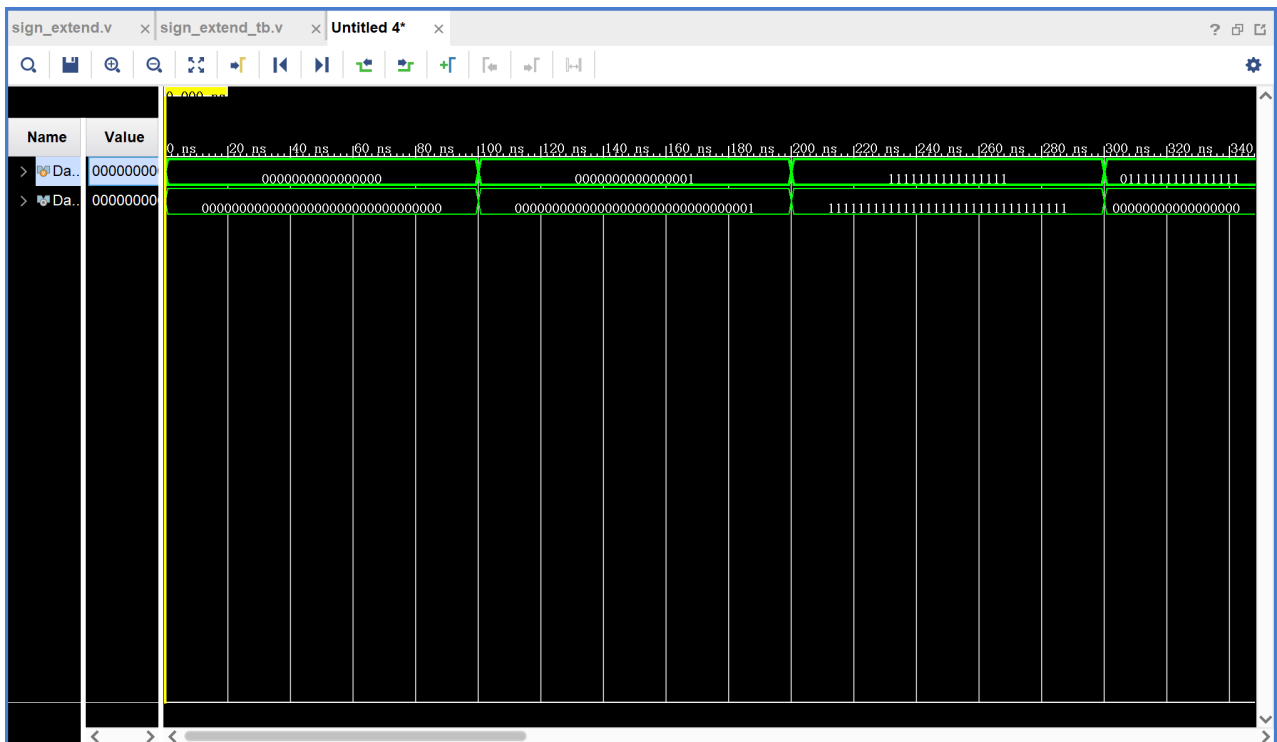


图 3: Sign Extend 功能仿真 1

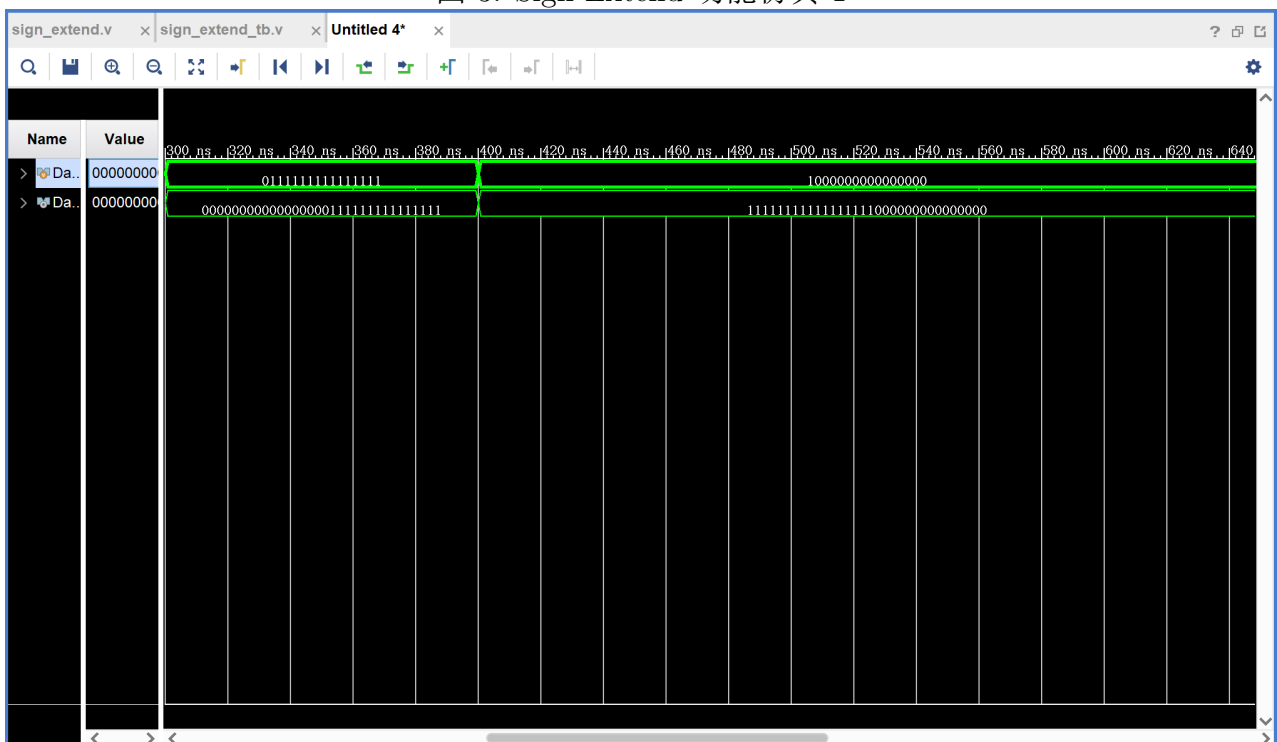


图 4: Sign Extend 功能仿真 2